# Computing - Home Assignment 3

Group 1

## Contents

**Exercise 48** **1**

**Exercise 53** **2**

**Exercise 62** **2**

**Exercise 67** **3**

## Exercise 48

Consider a floating-point number system with base $b$, precision $p$, and exponent range $[e_{\min}, e_{\max}]$.

(a) How many different non-zero normalized numbers are there?

Normalized numbers have the following general form:

$$x = \sigma(\delta_0 + \frac{\delta_1}{b^1} + \cdots + \frac{\delta_{p-1}}{b^{\,p-1}})b^{\,e}$$

where $e \in [e_{min}, e_{max}],\ i \in \{0, \ldots, p-1\},\ \delta_i \in \{0, \ldots, b-1\}$ and $\delta_0 \neq 0$.

For $\delta_0$ we choose from $b-1$ numbers (excl. 0), while for each of the remaining $p-1$ digits $\delta_1, \ldots, \delta_{p-1}$ we choose from $b$ numbers. For the exponent we choose from $e_{max} - e_{min} + 1$ integers and for the sign we have 2 possibilities. Hence, the total amount of numbers is obtained as

$$2 \cdot (b-1) \cdot b^{\,p-1} \cdot (e_{max} - e_{min} + 1)$$

Since we need normalized numbers, we have $\delta_0 \neq 0$ and hence all 0s are excluded. Moreover, zero cannot be normalized.

(b) What are the smallest (normalized) and largest positive numbers?

The smallest normalized positive number is

$$x_s = +(1 + \frac{0}{b^1} + \cdots + \frac{0}{b^{\,p-1}})b^{e_{min}}$$

The largest normalized positive number is

$$x_l = +((b-1) + \frac{b-1}{b^1} + \cdots + \frac{b-1}{b^{\,p-1}})b^{e_{max}} = b(1 - b^{-p})b^{e_{max}}$$

## Exercise 53

In many floating-point systems, a quick approximation to the unit roundoff error can be obtained by the approximation

$$\epsilon_{\text{mach}} \approx |3 \cdot (4/3 - 1) - 1|.$$

What is the idea behind this "trick"? Does it work when using IEEE 754 double precision?

In regular arithmetic this is 0, but in floating-point arithmetic it is not. Since 4/3 is an infinite (non-terminating) decimal ($1.333\ldots$), it is rounded to the nearest floating-point number in the given system.

Let float($x$) represent the floating-point variant of $x$. Then the floating-point representation of float($\frac{4}{3}$) = $\frac{4}{3} + \eta$ is given by $\frac{4}{3}$ and some rounding error $\eta$. Then we have

$$\begin{aligned}
\epsilon_{\text{mach}} &\approx \left|3(\text{float}\left(\frac{4}{3}\right) - 1) - 1\right| = \\
&= \left|3\left(\frac{4}{3} + \eta - 1\right) - 1\right| = \\
&= \left|3\left(\frac{1}{3} + \eta\right) - 1\right| = \\
&= |1 + 3\eta - 1| = \\
&= |3\eta|
\end{aligned}$$

This trick also works with IEEE 754 double precision. Consider $\frac{4}{3}$ in binary $\frac{4}{3} = (1.010101\ldots)_2$. In IEEE 754 we have 52 bits for the significand, hence the significand is $0101\ldots$ (the first bit is skipped). What gets discarded then is $(010101\ldots)_2 \times 2^{-52} = \frac{1}{3} \cdot 2^{-52}$. Hence, the floating-point representation of $\frac{4}{3}$ under IEEE 754 is

$$\text{float}\left(\frac{4}{3}\right) = \frac{4}{3} - \frac{1}{3} \cdot 2^{-52}$$

Similarly to the general example above, we have

$$\begin{aligned}
\epsilon_{mach} &\approx \left|3 * \left(\text{float}\left(\frac{4}{3}\right) - 1\right) - 1\right| = \\
&= \left|3 * \left(\frac{4}{3} - \frac{1}{3} \cdot 2^{-52} - 1\right) - 1\right| = \\
&= \left|3 * \left(\frac{1}{3} - \frac{1}{3} \cdot 2^{-52}\right) - 1\right| = \\
&= \left|1 - 2^{-52} - 1\right| = \\
&= 2^{-52}
\end{aligned}$$

## Exercise 62

Suppose $x$ and $y$ can be represented without error in double precision. Can the same be said for $x^2$ and $y^2$? Which would be more accurate, $x^2 - y^2$ or $(x+y)(x-y)$? For what values of $x$ and $y$, relative to each other, is there a substantial difference in the accuracy of the two expressions?

If $x$ and $y$ are represented without error in double precision, we can't be sure that $x^2$ or $y^2$ will also be represented without error. Suppose we have $x = 2^{52} + 1$. The double representation of this number will need 53 bits and its thus exactly representable ($\delta_0 = 1, \delta_i = 0$ for $i \in [1, 51], \delta_{52} = 1$). Then $x^2 = (2^{52} + 1)^2 =$

$2^{104} + 2^{53} + 1$. This number will require 105 bits to be represented exactly. Thus, the +1 will be rounded away.

The difference of squares is therefore more accurately obtained using $(x + y)(x - y)$. If we use $x^2 - y^2$, then the squares are rounded first with small relative errors introduced (up to $2^{-53}$). The subtraction of the squares cancels the leading digits and the relative error will dominate the result. This is also known as "catastrophic cancellation". If we use $(x + y)(x - y)$ instead, then the subtraction is applied using exact numbers (by assumption). The formula $(x + y)(x - y)$ also avoids overflow and underflow of $x^2$ and $y^2$ for very large or very small values of $x$ and $y$. The difference between the two formulas will be substantial when $x \approx y$.

## Exercise 67

According to Wilson's Theorem, a fail-safe test of whether a given number $n$ is prime or not is to check if $n$ divides $(n-1)! + 1$. If it does, it is sure to be prime, if not, it is a composite number. Write a function that implements a primality test based on Wilson's Theorem and test all the integers from 2 up to 25 for their primality using your function. Is this a good way to test for primality using a computer? For which number does it start to fail and why?

```
wilson_test <- function(n) {

  big_number <- factorial(n-1) + 1
  remainder  <- big_number %% n

  if(remainder == 0) {
    return(TRUE)
  } else {
    return(FALSE)
  }
}

integers <- seq(2,25)
for(i in integers) {
  cat(i, ":", wilson_test(i),"\n")
}
```

```
## 2 : TRUE
## 3 : TRUE
## 4 : FALSE
## 5 : TRUE
## 6 : FALSE
## 7 : TRUE
## 8 : FALSE
## 9 : FALSE
## 10 : FALSE
## 11 : TRUE
## 12 : FALSE
## 13 : TRUE
## 14 : FALSE
## 15 : FALSE
## 16 : FALSE
## 17 : TRUE
## 18 : FALSE
```

```
## 19 : TRUE
## 20 : TRUE
## 21 : TRUE
## 22 : TRUE
```

```
## Warning in wilson_test(i): probable complete loss of accuracy in modulus
```

```
## 23 : FALSE
```

```
## Warning in wilson_test(i): probable complete loss of accuracy in modulus
```

```
## 24 : TRUE
```

```
## Warning in wilson_test(i): probable complete loss of accuracy in modulus
```

```
## 25 : FALSE
```

We can see that for $n = 20$ we already have an incorrect result. The reason is that the factorial is represented as a "double" in R.

```
typeof(factorial(5))
```

```
## [1] "double"
```

This means that after certain $n$, this `factorial(n-1)` is no longer represented exactly. In fact, above $2^{53}$ the gap between consecutive doubles is at least 2. The Wilson's test depends on adding 1 to the factorial, so this $+1$ may be rounded up after certain $n$.

```
# Example for a double's step in R after 2^53
2^53+2 == 2^53
```

```
## [1] FALSE
```

```
2^53+1 == 2^53
```

```
## [1] TRUE
```

From the output of the function above we can see that the first wrong answer is at $n = 20$. This happens, because $(20 - 1)! > 2^{53}$. Then we have

```
factorial(20-1) + 1 == factorial(20-1)
```

```
## [1] TRUE
```

Then the answer is wrong because 19! is divisible by 20.

Moreover, $(n - 1)!$ grows faster than exponentially, which means that its computation is slow and overflows early. Hence, this is not a good approach to test if a number is prime with a computer.